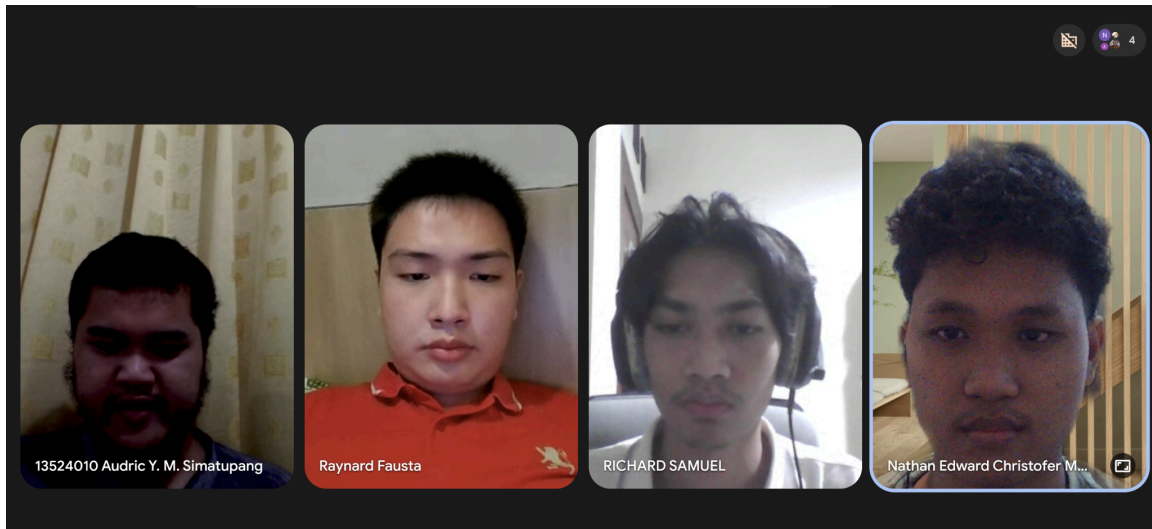


LAPORAN TUGAS BESAR
MATA KULIAH TEORI BAHASA FORMAL DAN OTOMATA
(IF2224)



TUGAS BESAR MILESTONE 2 : SYNTAX ANALYSIS
KELOMPOK SHT-K02

Anggota :

13524010 - Audric Yusuf Maynard Simatupang

13524052 - Raynard Fausta

13524062 - Nathan Edward Christoper Marpaung

13524112 - Richard Samuel Simanullang

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

LAPORAN TUGAS BESAR.....	1
DAFTAR ISI.....	2
Teori Singkat.....	0
A. Syntax Analysis.....	0
B. Context-Free Grammar.....	0
C. Recursive Descent Parsing.....	0
D. Parse Tree.....	1
Perancangan & Implementasi.....	2
Perancangan.....	2
A. Rancangan Arsitektur.....	2
B. Rancangan Grammar.....	5
Implementasi.....	14
A. Grammar.....	14
B. Parse Tree.....	16
C. Node.....	19
D. Parser.....	20
E. Main.....	28
Pengujian.....	30
Testing 1.....	30
Testing 2.....	33
Testing 3.....	40
Testing 4.....	49
Testing 5.....	56
Testing 6.....	57
Testing 7.....	58
Testing 8.....	59
Kesimpulan dan Saran.....	61
Kesimpulan.....	61
Saran.....	61
Lampiran.....	62
Link Repositori.....	62
Pembagian Tugas.....	62
Referensi.....	63

Teori Singkat

A. Syntax Analysis

Syntax Analysis adalah tahap lanjutan dalam pengembangan compiler setelah Lexical Analysis. Tahap ini terdiri pengecekan urutan token-token dari Lexical Analysis sesuai dengan aturan syntax dengan melakukan parsing. Tujuan dari Syntax Analysis adalah untuk memeriksa dan memastikan bahwa token input berada pada urutan yang benar sesuai dengan aturan syntax atau grammar dari language compiler tersebut.

B. Context-Free Grammar

Context-Free Grammar (CFG) adalah sebuah set aturan yang mendeskripsikan mengenai aturan syntax dari sebuah language. CFG mendeskripsikan sebuah uraian string sesuai dengan susunan statement atau ekspresi dari sebuah language. CFG terdiri dari 3 hal, yaitu simbol non-terminal (variabel), simbol terminal, dan aturan produksi.

Simbol terminal merupakan simbol atau token sebenarnya dari sebuah language tersebut. Simbol non-terminal merupakan simbol abstrak atau variabel yang merepresentasikan hubungan antar-simbol. Aturan produksi merupakan aturan dari sebuah language yang menjelaskan bagaimana hubungan sebuah simbol non-terminal dengan simbol non-terminal dan terminal lainnya, dan menjelaskan mengenai aturan memproduksi simbol non-terminal dan terminal yang berhubungan tersebut. Secara umum, CFG menjelaskan mengenai uraian string apa saja yang valid dan dapat diproduksi oleh sebuah language tersebut.

C. Recursive Descent Parsing

Parsing adalah proses untuk memahami struktur kalimat dan ekspresi yang dibaca berdasarkan aturan grammar yang ada pada suatu language. Dalam konteks tugas besar ini, parsing adalah proses untuk menganalisis token syntax yang diterima untuk membangun sebuah struktur kalimat berdasarkan grammar formal dari

language yang dianalisis. Parsing akan memverifikasi token input sesuai dengan aturan syntax yang sudah dibangun oleh language tersebut. Dalam melakukan parsing, terdapat 2 metode parsing yaitu top-down dan bottom-up. Top-down berarti urutan struktur parsing dimulai dari atas ke bawah, dan dari kiri ke kanan.

Salah satu bentuk top-down parsing adalah Recursive Descent Parsing. Recursive Descent Parsing adalah metode melakukan parsing dengan membangun beberapa prosedur rekursif sesuai dengan syntax atau grammar language tersebut, dan dijalankan hingga mendapatkan susunan elemen yang sesuai dengan grammar tersebut. Fungsi rekursif tersebut akan dijalankan secara backtracking, di mana jika fungsi menemukan susunan ekspresi yang tidak sesuai grammar ataupun token input, maka fungsi akan backtrack menuju rekursi sebelumnya dan lanjut menguji susunan token dan grammar lainnya. Hal ini diulang hingga fungsi berhasil membangun sebuah parse tree yang sesuai dengan token input dan grammar language. Untuk mencegah terjadinya infinite looping, maka rancangan produksi grammar harus dimulai dari simbol terminal ataupun simbol variabel yang tidak memproduksi dirinya sendiri.

D. Parse Tree

Parse tree adalah bentuk grafik representasi grafik dari uraian susunan token sesuai dengan grammar language yang dianalisis. Root node dari parse tree akan menandakan start symbol dari uraian token/ekspresi tersebut, dengan node anak dari masing-masing node menandakan uraian atau produksi dari grammar language tersebut. Node dari sebuah tree akan terdiri dari node variabel yang akan melakukan produksi node baru sesuai dengan grammar dari language tersebut, ataupun simbol terminal yang akan berfungsi sebagai leaf node parse tree tersebut.

Perancangan & Implementasi

Perancangan

A. Rancangan Arsitektur

Program pada Milestone 2 dirancang sebagai lanjutan dari lexical analyzer pada Milestone 1 dengan menambahkan modul syntax analyzer/parser. Melanjutkan arsitektur yang sudah ditentukan pada Milestone 1, teknologi yang digunakan pada Milestone 2 adalah bahasa C dengan standar kompilasi C11 melalui gcc.

Program disusun secara modular di dalam folder src, dengan modul utama: lexer untuk menghasilkan token dari source code Arion, token untuk mendefinisikan jenis token, parser untuk melakukan syntax analysis, parse_tree untuk membangun dan mencetak pohon parse, grammar untuk validasi aturan produksi, serta main.c sebagai pengatur alur program dan interaksi pengguna.

Untuk Milestone 2, komponen terpenting adalah parser.c dan parser.h. Parser.h menyediakan interface berupa SyntaxResult, analyzeSyntaxFile, dan freeSyntaxResult, sedangkan parser.c mengimplementasikan algoritma Recursive Descent melalui fungsi-fungsi seperti parseProgram, parseDeclarationPart, parseStatement, parseExpression, dan fungsi parser lain yang masing-masing merepresentasikan satu aturan grammar.

Alur pekerjaan kode parser adalah sebagai berikut :

1. Inisialisasi Struktur Parser

Proses analisis sintaks dimulai dengan menetapkan kondisi awal pada struktur Parser, yaitu inisialisasi ke nilai netral: array token bernilai NULL, count dan capacity bernilai nol, cursor posisi (pos) bernilai nol, flag fatalError bernilai false, serta buffer error dikosongkan.

2. Pemuatan Aliran Token dari Lexer

Setelah inisialisasi selesai, parser memanggil fungsi loadTokensFromLexer yang berperan sebagai jembatan antara analisis

leksikal dan analisis sintaks. Fungsi ini melakukan inisialisasi lexer yang berlanjut secara iteratif hingga ditemukan token bertipe TOKEN_EOF.

3. ParseProgram

Setelah seluruh token termuat, parser memanggil parseProgram sebagai axiom. Fungsi ini melaksanakan parsing terhadap empat komponen utama secara berurutan:

1. <program-header> yang terdiri atas kata kunci program, identifier nama program, dan token titik koma(;)
2. <declaration-part> yang menampung deklarasi konstanta, tipe, variabel, serta subprogram
3. <compound-statement> yang merupakan blok begin..end berisi badan utama program
4. Token terminasi, yaitu period(.)

4. Parsing Bagian Deklarasi

Fungsi parseDeclarationPart memparse empat jenis deklarasi sesuai urutan yang ditentukan, yaitu deklarasi konstanta (const), deklarasi tipe (type), deklarasi variabel (var), dan deklarasi subprogram (procedure atau function).

5. Parsing Pernyataan melalui Mekanisme Dispatch

Pada statement, fungsi parseStatement berfungsi sebagai dispatcher yang memilih sub-parser berdasarkan token awal pernyataan, dengan pemetaan sebagai berikut:

1. TOKEN_IDENT diarahkan ke parseAssignmentStatement apabila lookahead menunjukkan adanya operator :=, atau ke parseProcedureFunctionCall apabila tidak
2. TOKEN_IFSY diarahkan ke parseIfStatement
3. TOKEN_CASESY diarahkan ke parseCaseStatement
4. TOKEN_WHILESY diarahkan ke parseWhileStatement
5. TOKEN_REPEATSY diarahkan ke parseRepeatStatement
6. TOKEN_FORSY diarahkan ke parseForStatement
7. TOKEN_BEGINSY diarahkan ke parseCompoundStatement

6. Parsing Ekspresi dengan Hierarki Presedensi

Untuk parsing ekspresi, parser menerapkan teknik precedence climbing dengan empat tingkat hierarki yang saling memanggil secara rekursif:

1. `parseExpression` menangani operator relasional (`=`, `<>`, `>`, `>=`, `<`, `<=`)
2. `parseSimpleExpression` menangani operator aditif (`+`, `-`, `or`) serta operator unary `+/-`
3. `parseTerm` menangani operator multiplikatif (`*`, `/`, `div`, `mod`, `and`)
4. `parseFactor` menangani atom ekspresi, yaitu identifier, konstanta literal, ekspresi berkurung, dan operator not

7. Pencocokan Token dan Konstruksi Parse Tree

Pada setiap pemanggilan sub-parser, parser melakukan pencocokan token melalui dua fungsi utama:

1. Fungsi `parserExpectToken` menerapkan kontrak ketat, yaitu apabila token saat ini sesuai dengan tipe yang diharapkan, token tersebut ditambahkan sebagai daun parse tree melalui `parserCreateTokenNode` dan cursor dimajukan ke posisi berikutnya
2. Fungsi `parserAcceptToken` hanya mencocokkan token jika sesuai dan tidak mencatat kesalahan apabila tidak cocok. Fungsi ini digunakan untuk konstruksi opsional seperti klausa `else` pada `if-statement`. Selama proses ini, node internal dibangun dengan label sesuai non-terminal yang sedang diparse

8. Informasi dan Penanganan Kesalahan

Apabila pencocokan token gagal, parser tidak langsung berhenti melainkan menerapkan:

1. `Panic Mode Recovery`, yang diimplementasikan melalui tiga fungsi sinkronisasi, yaitu `parserSkipUntilStatementBoundary`, `parserSkipUntilExpressionBoundary`, dan `parserSkipUntilTypeBoundary`. Fungsi-fungsi ini bekerja dengan

melompati token sampai mencapai batas konstruksi yang dikenali sebagai titik aman untuk melanjutkan parsing

2. Token Insertion, yang diimplementasikan melalui `parserShouldInsertMissingToken`. Fungsi ini menentukan secara heuristik apakah token yang hilang dapat dianggap implisit berdasarkan konteks token saat ini

Setiap kesalahan dicatat melalui `parserRecordSyntaxErrorV` dengan format `"Line {nomor_baris} near {deskripsi_token}: {detail_pesan}"` dan disimpan ke dalam buffer error.

9. Validasi terhadap Tata Bahasa Formal

Setelah seluruh proses parsing selesai dan tidak ditemukan kesalahan, parser memanggil fungsi `validateWithGrammar` sebagai langkah verifikasi akhir. Tahap ini terdiri atas 2 bagian:

1. Fungsi `convertParseTreeToGrammarNode` melakukan transformasi parse tree internal menjadi struktur Node yang dipahami oleh modul grammar
2. Fungsi `is_program_complete` dipanggil untuk memverifikasi bahwa pohon yang dihasilkan benar-benar memenuhi seluruh aturan produksi formal

Apabila validasi gagal, parser mencatat pesan "Grammar validation failed: parse tree does not match the grammar rules."

B. Rancangan Grammar

Perancangan grammar yang digunakan untuk milestone 2 mengacu pada aturan produksi yang telah disediakan pada spek. Berikut merupakan tabel yang memetakan setiap production rule pada grammar dengan fungsi parser yang mengimplementasikannya dalam kode program :

Produksi	Fungsi	Penjelasan
program -> program-header	parseProgram	Memanggil

declaration-part compound-statement period		parseProgramHeader, lalu parseDeclarationPart, lalu parseCompoundStatement, kemudian mengecek token period dan EOF.
program-header -> programsy ident semicolon	parseProgramHeader	Mengecek urutan token programsy, ident, dan semicolon.
declaration-part -> const-declaration* type-declaration* var-declaration* subprogram-declaration*	parseDeclarationPart	Memakai beberapa while untuk membaca deklarasi const, type, var, dan procedure/function secara berulang.
const-declaration -> constsy (ident eql constant semicolon)+	parseConstDeclaration	Setelah constsy, fungsi membaca pasangan ident eql constant semicolon selama token berikutnya masih ident.
constant -> charcon string (plus minus)? (ident intcon realcon)	parseConstant	Mengecek konstanta charcon/string terlebih dahulu, lalu menangani tanda plus/minus opsional sebelum ident, intcon, atau realcon.
type-declaration -> typesy (ident eql type semicolon)+	parseTypeDeclaration	Setelah typesy, fungsi membaca ident eql type semicolon berulang dengan memanggil parseType.
var-declaration -> varsy (identifier-list colon type semicolon)+	parseVarDeclaration	Membaca identifier-list, lalu colon, lalu memanggil parseType, kemudian

		mengecek semicolon.
identifier-list -> ident (comma ident)*	parseIdentifierList	Mengecek satu ident, lalu selama ada comma, mengecek ident berikutnya.
type -> ident array-type range enumerated record-type	parseType	Memilih cabang berdasarkan token sekarang: arraysy ke parseArrayType, recordsy ke parseRecordType, tanda kurung ke parseEnumerated, range ke parseRange, atau langsung ident.
array-type -> arraysy lbrack (range ident) rbrack ofsy type	parseArrayType	Mengecek arraysy dan lbrack, lalu membaca range atau ident, kemudian rbrack, ofsy, dan memanggil parseType.
range -> constant period period constant	parseRange	Memanggil parseConstant, mengecek dua token period, lalu memanggil parseConstant lagi.
enumerated -> lparent ident (comma ident)* rparent	parseEnumerated	Mengecek lparent, membaca ident, lalu membaca pasangan comma ident secara berulang, dan ditutup dengan rparent.
record-type -> recordsy field-list endsy	parseRecordType	Mengecek recordsy, memanggil parseFieldList jika belum bertemu endsy, lalu mengecek endsy.

field-list -> field-part (semicolon field-part)*	parseFieldList	Memanggil parseFieldPart, lalu membaca semicolon field-part secara berulang.
field-part -> identifier-list colon type	parseFieldPart	Memanggil parseIdentifierList, mengecek colon, lalu memanggil parseType.
subprogram-declaration -> procedure-declaration function-declaration	parseSubprogramDeclaration	Jika token sekarang proceduresy, memanggil parseProcedureDeclaration. Jika functionsy, memanggil parseFunctionDeclaration.
procedure-declaration -> proceduresy ident formal-parameter-list? semicolon block semicolon	parseProcedureDeclaration	Mengecek proceduresy ident, membaca parameter formal jika ada lparent, lalu mengecek semicolon, memanggil parseBlock, dan menutup dengan semicolon.
function-declaration -> functionsy ident formal-parameter-list? colon ident semicolon block semicolon	parseFunctionDeclaration	Mengecek functionsy ident, membaca parameter formal jika ada, lalu mengecek colon ident semicolon, memanggil parseBlock, dan menutup dengan semicolon.
block -> declaration-part compound-statement	parseBlock	Memanggil parseDeclarationPart, lalu parseCompoundStatement.
formal-parameter-list -> lparent parameter-group	parseFormalParameterList	Mengecek lparent, memanggil

(semicolon parameter-group)* rparent		parseParameterGroup, lalu membaca semicolon parameter-group secara berulang sampai rparent.
parameter-group -> identifier-list colon (ident array-type)	parseParameterGroup	Memanggil parseIdentifierList, mengecek colon, lalu membaca array-type jika token arraysy, atau membaca ident sebagai tipe biasa.
compound-statement -> beginsy statement-list endsy	parseCompoundStatement	Mengecek beginsy, memanggil parseStatementList jika belum langsung endsy, lalu mengecek endsy.
statement-list -> statement (semicolon statement)*	parseStatementList	Memanggil parseStatement, lalu selama belum terminator, membaca semicolon dan statement berikutnya.
statement -> assignment-statement if-statement case-statement while-statement repeat-statement for-statement procedure/function-call compound-statement	parseStatement	Memilih jenis statement berdasarkan token awal. Untuk ident, fungsi membedakan assignment atau procedure/function call memakai hasBecomesAfterVariable.
variable -> ident component-variable*	parseVariable	Mengecek ident, lalu selama ada lbrack atau period,

		memanggil parseComponentVariable.
component-variable -> lbrack index-list rbrack period ident	parseComponentVariable	Jika token lbrack, fungsi membaca index-list lalu rbrack. Jika token period, fungsi membaca period ident.
index-list -> (intcon charcon ident) (comma index-list)*	parseIndexList	Mengecek index berupa intcon, charcon, atau ident. Jika ada comma, fungsi memanggil parseIndexList lagi secara rekursif.
assignment-statement -> variable becomes expression	parseAssignmentStatement	Memanggil parseVariable, mengecek becomes, lalu memanggil parseExpression.
if-statement -> ifsy expression thensy statement (elsy statement)?	parseIfStatement	Mengecek ifsy, memanggil parseExpression, mengecek thensy, memanggil parseStatement, lalu membaca bagian elsy statement jika ada.
case-statement -> casesy expression ofsy case-block endsy	parseCaseStatement	Mengecek casesy, memanggil parseExpression, mengecek ofsy, memanggil parseCaseBlock, lalu mengecek endsy.
case-block -> constant (comma constant)* colon statement	parseCaseBlock	Memanggil parseConstant, membaca tambahan comma constant jika ada, mengecek colon, lalu memanggil

		parseStatement.
while-statement -> whilesy expression dosy statement	parseWhileStatement	Mengecek whilesy, memanggil parseExpression, mengecek dosy, lalu memanggil parseStatement.
repeat-statement -> repeatsy statement-list untilsy expression	parseRepeatStatement	Mengecek repeatsy, memanggil parseStatementList, mengecek untilsy, lalu memanggil parseExpression.
for-statement -> forsy ident becomes expression (tosy downtosy) expression dosy statement	parseForStatement	Mengecek forsy ident becomes, memanggil parseExpression, mengecek tosy atau downtosy, memanggil parseExpression lagi, mengecek dosy, lalu memanggil parseStatement.
procedure/function-call -> ident (lparent parameter-list? rparent)?	parseProcedureFunctionCall	Mengecek ident. Jika ada lparent, fungsi membaca parameter-list jika tidak langsung rparent, lalu mengecek rparent.
parameter-list -> expression (comma expression)*	parseParameterList	Memanggil parseExpression, lalu selama ada comma, memanggil parseExpression lagi untuk parameter berikutnya.
expression -> simple-expression	parseExpression	Memanggil parseSimpleExpression, lalu

(relational-operator simple-expression)?		jika ada operator relasional, membaca operator tersebut dan memanggil parseSimpleExpression lagi.
simple-expression -> (plus minus)? term (additive-operator term)*	parseSimpleExpression	Membaca tanda unary plus/minus jika ada, memanggil parseTerm, lalu selama ada operator aditif, membaca operator dan memanggil parseTerm lagi.
term -> factor (multiplicative-operator factor)*	parseTerm	Memanggil parseFactor, lalu selama ada operator multiplikatif, membaca operator dan memanggil parseFactor lagi.
factor -> ident intcon realcon charcon string lparent expression rparent notsy factor procedure/function-call variable	parseFactor	Jika ident diikuti lparent, memanggil parseProcedureFunctionCall. Jika ident diikuti akses array/record, memanggil parseVariable. Konstanta dibaca langsung, tanda kurung memanggil parseExpression, notsy memanggil parseFactor lagi.
relational-operator -> eql neq gtr geq lss leq	isRelationalOperator, used in parseExpression	Mengecek apakah token termasuk eql, neq, gtr, geq, lss, atau leq. Digunakan oleh parseExpression.

additive-operator -> plus minus orsy	isAdditiveOperator, used in parseSimpleExpression	Mengecek apakah token termasuk plus, minus, atau orsy. Digunakan oleh parseSimpleExpression.
multiplicative-operator -> times rdiv idiv imod andsy	isMultiplicativeOperator, used in parseTerm	Mengecek apakah token termasuk times, rdiv, idiv, imod, atau andsy. Digunakan oleh parseTerm.

Implementasi

A. Grammar

Rancangan grammar pada tahap perancangan kemudian di implementasi ke dalam program. Sebuah linkedlist yang berisi token-token hasil lexical analysis akan digunakan untuk pemeriksaan sesuai grammar hingga mencapai node terakhir. Untuk simbol terminal, pengecekan akan dilakukan dengan mengecek apakah node yang dibaca tidak NULL dan memiliki konten yang sesuai dengan token terminal. Untuk grammar produksi oleh variabel, node akan melakukan pengecekan pada simbol terminal dan melakukan proses rekursif jika bertemu variabel lainnya. Fungsi akan memeriksa semua kemungkinan dari variabel secara rekursif, dan akan melakukan backtracking jika terdapat kesalahan yang terdeteksi di dalam rekursif. Keseluruhan kode ini disimpan di dalam file grammar.c dan grammar.hpp. Contoh kode produksi grammar dan pengecekan simbol terminal adalah:

```
#include "grammar.h"
#include <string.h>

//contoh pengecekan simbol terminal

bool isExactLabel(Node *node, const char *label) {
    return node!=NULL && strcmp(node->name, label)==0;
}

bool isIdentLabel(Node *node) {
    return node!=NULL && strncmp(node->name, "ident", 5)==0;
}

bool isIntconLabel(Node *node) {
    return node!=NULL && strncmp(node->name, "intcon", 6)==0;
}

bool isRealconLabel(Node *node) {
    return node!=NULL && strncmp(node->name, "realcon", 7)==0;
}

bool isCharconLabel(Node *node) {
    return node!=NULL && strncmp(node->name, "charcon", 7)==0;
}

bool isStringLabel(Node *node) {
    return node!=NULL && strncmp(node->name, "string", 6)==0;
}

//contoh grammar produksi
```

```

bool isSemicolonStatementPair(Node *semicolonNode, Node *statementNode)
{
    return isExactLabel(semicolonNode, "semicolon") &&
is_statement_complete(statementNode);
}

bool isCommaExpressionPair(Node *commaNode, Node *exprNode) {
    return isExactLabel(commaNode, "comma") &&
is_expression_complete(exprNode);
}

bool isCommaIdentPair(Node *commaNode, Node *identNode) {
    return isExactLabel(commaNode, "comma") && isIdentLabel(identNode);
}

bool isSemicolonParameterGroupPair(Node *semicolonNode, Node
*parameterGroupNode) {
    return isExactLabel(semicolonNode, "semicolon") &&
is_parameter_group_complete(parameterGroupNode);
}

bool isSemicolonFieldPartPair(Node *semicolonNode, Node *fieldPartNode)
{
    return isExactLabel(semicolonNode, "semicolon") &&
is_field_part_complete(fieldPartNode);
}

bool isSemicolonCaseBlockPair(Node *semicolonNode, Node *caseBlockNode)
{
    return isExactLabel(semicolonNode, "semicolon") &&
is_case_block_complete(caseBlockNode);
}

bool isConstDeclEntry(Node *identNode, Node *eqlNode, Node
*constantNode, Node *semicolonNode) {
    return isIdentLabel(identNode) &&
isExactLabel(eqlNode, "eql") &&
is_constant_complete(constantNode) &&
isExactLabel(semicolonNode, "semicolon");
}

bool isTypeDeclEntry(Node *identNode, Node *eqlNode, Node *typeNode,
Node *semicolonNode) {
    return isIdentLabel(identNode) &&
isExactLabel(eqlNode, "eql") &&
is_type_complete(typeNode) &&
isExactLabel(semicolonNode, "semicolon");
}

bool isVarDeclEntry(Node *identifierListNode, Node *colonNode, Node
*typeNode, Node *semicolonNode) {
    return is_identifier_list_complete(identifierListNode) &&
isExactLabel(colonNode, "colon") &&
is_type_complete(typeNode) &&
isExactLabel(semicolonNode, "semicolon");
}

```

```

}

// 1
bool is_program_complete(Node *node) {
    if (!isExactLabel(node, "<program>")) return false;
    if (node->childCount!=4) return false;
    if (!is_program_header_complete(node->child[0])) return false;
    if (!is_declaration_part_complete(node->child[1])) return false;
    if (!is_compound_statement_complete(node->child[2])) return false;
    return isExactLabel(node->child[3], "period");
}

```

B. Parse Tree

ADT Parse Tree diimplementasikan untuk menyimpan informasi hasil parsing dan urutan struktur token yang benar dan sesuai grammar. Parse Tree yang diimplementasi merupakan sebuah LinkedList Tree yang terdiri atas ParseTreeNode. Setiap ParseTreeNode akan menyimpan informasi token pada char *label, pointer pada child ParseTreeNode di **children, dan jumlah child yang dimiliki oleh sebuah node dan kapasitasnya pada childCount dan childCapacity.

```

typedef struct ParseTreeNode {
    char *label; // Label atau nama node (string)
    struct ParseTreeNode **children; // Array dinamis dari pointer ke
    anak node
    size_t childCount; // Jumlah anak node saat ini
    size_t childCapacity; // Kapasitas maksimal array anak
    node
} ParseTreeNode;

```

ADT Parse Tree ini akan digunakan oleh syntax analysis untuk menyimpan output hasil syntax analysis. Untuk membantu pembuatan ADT Parse Tree, beberapa fungsi helper juga diimplementasikan ke dalam file parse_tree.c. ParseTreeCreateNode akan membuat node baru dengan label yang dimasukkan sebagai parameter, parseTreeAddChild akan melakukan alokasi memori untuk menambahkan node baru dari parameter fungsi sebagai daun dari parse_tree, dan parseTreeFree akan melakukan dealokasi memori setelah Syntax Analysis selesai dan hasil parse tree ingin dihapus dari memori.

```

ParseTreeNode *parseTreeCreateNode(const char *label) {
    ParseTreeNode *node;

```

```

    if (label==NULL) {
        return NULL;
    }

    node=(ParseTreeNode *)malloc(sizeof(ParseTreeNode));
    if (node==NULL) {
        return NULL;
    }

    node->label=cloneString(label);
    if (node->label==NULL) {
        free(node);
        return NULL;
    }

    node->children=NULL;
    node->childCount=0;
    node->childCapacity=0;
    return node;
}

bool parseTreeAddChild(ParseTreeNode *parent, ParseTreeNode *child) {
    ParseTreeNode **newChildren;
    size_t newCapacity;

    if (parent==NULL || child==NULL) {
        return false;
    }

    if (parent->childCount==parent->childCapacity) {
        newCapacity=parent->childCapacity==0 ? 4 : parent->childCapacity
* 2;
        newChildren=(ParseTreeNode **)realloc(parent->children,
newCapacity * sizeof(ParseTreeNode *));
        if (newChildren==NULL) {
            return false;
        }

        parent->children=newChildren;
        parent->childCapacity=newCapacity;
    }

    parent->children[parent->childCount]=child;
    parent->childCount++;
    return true;
}

void parseTreeFree(ParseTreeNode *root) {
    size_t i;

    if (root==NULL) {
        return;
    }

    for (i=0; i < root->childCount; i++) {
        parseTreeFree(root->children[i]);
    }
}

```

```

    }

    free(root->children);
    free(root->label);
    free(root);
}

```

File `parse_tree.c` juga mengimplementasikan fungsi untuk hasil parse tree dari syntax analysis dalam bentuk pohon pada output CLI. Fungsi ini dilakukan lewat `parseTreePrint` dan `parseTreePrintChildren` yang akan melakukan print secara rekursif dari root node hingga leaf node, dan mengoutput hasil rekursif sesuai dengan format print yang sudah dibentuk.

```

#include "parse_tree.h"

#include <stdlib.h>
#include <string.h>

#define TREE_BRANCH "|-- "
#define TREE_LAST "`-- "
#define TREE_CONT "|  "
#define TREE_SPACE "    "

static char *cloneString(const char *src) {
    char *copy;
    size_t len;

    if (src==NULL) {
        return NULL;
    }

    len=strlen(src);
    copy=(char *)malloc(len + 1);
    if (copy==NULL) {
        return NULL;
    }

    memcpy(copy, src, len + 1);
    return copy;
}

static char *makeNextPrefix(const char *prefix, bool isLastChild) {
    const char *addition;
    char *newPrefix;
    size_t prefixLen;
    size_t additionLen;

    addition=isLastChild ? TREE_SPACE : TREE_CONT;

```

```

    prefixLen=strlen(prefix);
    additionLen=strlen(addition);

    newPrefix=(char *)malloc(prefixLen + additionLen + 1);
    if (newPrefix==NULL) {
        return NULL;
    }

    memcpy(newPrefix, prefix, prefixLen);
    memcpy(newPrefix + prefixLen, addition, additionLen + 1);
    return newPrefix;
}

static void parseTreePrintChildren(const ParseTreeNode *node, FILE
*stream, const char *prefix) {
    size_t i;

    for (i=0; i < node->childCount; i++) {
        const ParseTreeNode *child=node->children[i];
        bool isLast=(i + 1)==node->childCount;
        char *nextPrefix;

        fprintf(stream, "%s%s%s\n", prefix, isLast ? TREE_LAST :
TREE_BRANCH, child->label);
        nextPrefix=makeNextPrefix(prefix, isLast);
        if (nextPrefix==NULL) {
            return;
        }

        parseTreePrintChildren(child, stream, nextPrefix);
        free(nextPrefix);
    }
}

void parseTreePrint(const ParseTreeNode *root, FILE *stream) {
    if (root==NULL || stream==NULL) {
        return;
    }

    fprintf(stream, "%s\n", root->label);
    parseTreePrintChildren(root, stream, "");
}

```

C. Node

Node adalah struktur ADT LinkedList yang berfungsi untuk menyimpan hasil token dari lexical analysis dan digunakan untuk syntax analysis sebagai urutan token yang akan diperiksa validitasnya. Node ini terdiri char *name yang akan menyimpan token yang perlu diperiksa, Node child yang merupakan list token selanjutnya, dan

childCount serta childCapacity untuk menghitung banyak children dan kapasitas children yang dimiliki. ADT ini disimpan di dalam file node.h.

```
#ifndef NODE_H
#define NODE_H

typedef struct Node {
    char *name;
    struct Node **child;
    int childCount;
    int childCapacity;
} Node;

#endif
```

D. Parser

Parser pada program ini merupakan komponen syntax analysis yang bertugas memvalidasi deretan token hasil lexer dan membentuk parse tree sesuai grammar yang sudah ditentukan. Pada implementasi ini, parser dibangun sebagai recursive descent parser, yaitu parser yang memetakan setiap aturan grammar ke dalam fungsi-fungsi parsing yang saling memanggil. Cuplikan kode untuk setiap langkah alur parsing adalah sebagai berikut :

1. Inisialisasi Struktur Parser

```
bool analyzeSyntaxFile(const char *inputPath, SyntaxResult *result) {
    Parser parser;
    ParseTreeNode *root;

    if (result==NULL || inputPath==NULL) {
        return false;
    }

    result->success=false;
    result->tree=NULL;
    result->message[0]='\0';

    parser.tokens=NULL;
    parser.count=0;
    parser.capacity=0;
    parser.pos=0;
    parser.fatalError=false;
    parser.errorCount=0;
    parser.errorLength=0;
    parser.error[0]='\0';

    if (!loadTokensFromLexer(&parser, inputPath)) {
        (void) snprintf(
```

```

        result->message,
        sizeof(result->message),
        "%s",
        parser.error[0]!='\0' ? parser.error : "Invalid syntax."
    );
    parserDestroy(&parser);
    return false;
}

root=parseProgram(&parser);
if (parser.fatalError || root==NULL) {
    (void)snprintf(
        result->message,
        sizeof(result->message),
        "%s",
        parser.error[0]!='\0' ? parser.error : "Invalid syntax."
    );
    parserDestroy(&parser);
    return false;
}

if (parser.errorCount > 0) {
    const char *prefix="Ditemukan error syntax:\n";
    size_t prefixLen=strlen(prefix);

    parseTreeFree(root);
    memcpy(result->message, prefix, prefixLen);
    result->message[prefixLen]='\0';
    if (prefixLen < sizeof(result->message) - 1) {
        size_t available=sizeof(result->message) - 1 - prefixLen;
        (void)strncat(result->message, parser.error, available);
    }
    parserDestroy(&parser);
    return false;
}

if (!validateWithGrammar(&parser, root)) {
    parseTreeFree(root);
    (void)snprintf(
        result->message,
        sizeof(result->message),
        "%s",
        parser.error[0]!='\0' ? parser.error : "Invalid syntax."
    );
    parserDestroy(&parser);
    return false;
}

result->success=true;
result->tree=root;
(void)snprintf(result->message, sizeof(result->message), "Syntax
analysis successful.");

    parserDestroy(&parser);
    return true;
}

```


2. Pemuatan Aliran Token dari Lexer

```
static bool loadTokensFromLexer(Parser *ps, const char *inputPath) {
    Lexer lx;

    initLexer(&lx, inputPath);
    if (!lx.ready) {
        return parserSetError(ps, "Failed to open source file: %s",
inputPath);
    }

    while (true) {
        Token tk=getToken(&lx);

        if (tk.type==TOKEN_COMMENT) {
            continue;
        }

        if (tk.type==TOKEN_UNKNOWN) {
            parserRecordSyntaxErrorToken(ps, &tk, "Invalid token.");
            continue;
        }

        if (!parserAppendToken(ps, &tk)) {
            closeLexer(&lx);
            return false;
        }

        if (tk.type==TOKEN_EOF) {
            break;
        }
    }

    closeLexer(&lx);
    return true;
}
```

3. ParseProgram

```
static ParseTreeNode *parseProgram(Parser *ps) {
    ParseTreeNode *node=parserCreateNode(ps, "<program>");
    ParseTreeNode *child;

    if (node==NULL) {
        return NULL;
    }

    child=parseProgramHeader(ps);
    if (child==NULL || !parserAttachChild(ps, node, child)) {
        parseTreeFree(node);
        return NULL;
    }
}
```

```

    child=parseDeclarationPart(ps);
    if (child==NULL || !parserAttachChild(ps, node, child)) {
        parseTreeFree(node);
        return NULL;
    }

    child=parseCompoundStatement(ps);
    if (child==NULL || !parserAttachChild(ps, node, child)) {
        parseTreeFree(node);
        return NULL;
    }

    if (!parserExpectToken(ps, TOKEN_PERIOD, node) ||
        !parserExpectToken(ps, TOKEN_EOF, NULL)) {
        parseTreeFree(node);
        return NULL;
    }

    return node;
}

```

4. Parsing Bagian Deklarasi

```

static ParseTreeNode *parseDeclarationPart(Parser *ps) {
    ParseTreeNode *node=parserCreateNode(ps, "<declaration-part>");

    if (node==NULL) {
        return NULL;
    }

    while (parserCurrentType(ps)==TOKEN_CONSTSY) {
        ParseTreeNode *constNode=parseConstDeclaration(ps);
        if (constNode==NULL || !parserAttachChild(ps, node, constNode)) {
            parseTreeFree(node);
            return NULL;
        }
    }

    while (parserCurrentType(ps)==TOKEN_TYPESY) {
        ParseTreeNode *typeNode=parseTypeDeclaration(ps);
        if (typeNode==NULL || !parserAttachChild(ps, node, typeNode)) {
            parseTreeFree(node);
            return NULL;
        }
    }

    while (parserCurrentType(ps)==TOKEN_VARSY) {
        ParseTreeNode *varNode=parseVarDeclaration(ps);
        if (varNode==NULL || !parserAttachChild(ps, node, varNode)) {
            parseTreeFree(node);
            return NULL;
        }
    }
}

```

```

    }

    while (parserCurrentType(ps)==TOKEN_PROCEDESYSY ||
           parserCurrentType(ps)==TOKEN_FUNCTIONSY) {
        ParseTreeNode *subprogramNode=parseSubprogramDeclaration(ps);
        if (subprogramNode==NULL || !parserAttachChild(ps, node,
subprogramNode)) {
            parseTreeFree(node);
            return NULL;
        }
    }

    return node;
}

```

5. Parsing Pernyataan melalui Mekanisme Dispatch

```

static ParseTreeNode *parseStatement(Parser *ps) {
    ParseTreeNode *node=parserCreateNode(ps, "<statement>");
    ParseTreeNode *child;
    TokenType current=parserCurrentType(ps);

    if (node==NULL) {
        return NULL;
    }

    switch (current) {
        case TOKEN_IDENT:
            if (hasBecomesAfterVariable(ps)) {
                child=parseAssignmentStatement(ps);
            }
            else {
                child=parseProcedureFunctionCall(ps);
            }
            break;
        case TOKEN_IFSY:
            child=parseIfStatement(ps);
            break;
        case TOKEN_CASESY:
            child=parseCaseStatement(ps);
            break;
        case TOKEN_WHILESY:
            child=parseWhileStatement(ps);
            break;
        case TOKEN_REPEATSY:
            child=parseRepeatStatement(ps);
            break;
        case TOKEN_FORSY:
            child=parseForStatement(ps);
            break;
        case TOKEN_BEGINSY:
            child=parseCompoundStatement(ps);
            break;
        default:

```

```

        parserRecordSyntaxErrorCurrent(ps, "Expected statement.");
        parserSkipUntilStatementBoundary(ps);
        return node;
    }

    if (child==NULL || !parserAttachChild(ps, node, child)) {
        parseTreeFree(node);
        return NULL;
    }

    return node;
}

```

6. Parsing Ekspresi dengan Hierarki Presedensi

```

static ParseTreeNode *parseExpression(Parser *ps) {
    ParseTreeNode *node=parserCreateNode(ps, "<expression>");
    ParseTreeNode *simpleExpr;

    if (node==NULL) {
        return NULL;
    }

    simpleExpr=parseSimpleExpression(ps);
    if (simpleExpr==NULL || !parserAttachChild(ps, node, simpleExpr)) {
        parseTreeFree(node);
        return NULL;
    }

    if (isRelationalOperator(parserCurrentType(ps))) {
        ParseTreeNode *opNode=parserCreateNode(ps,
"<relational-operator>");
        TokenType op=parserCurrentType(ps);

        if (opNode==NULL || !parserExpectToken(ps, op, opNode)) {
            parseTreeFree(opNode);
            parseTreeFree(node);
            return NULL;
        }

        if (!parserAttachChild(ps, node, opNode)) {
            parseTreeFree(node);
            return NULL;
        }

        simpleExpr=parseSimpleExpression(ps);
        if (simpleExpr==NULL || !parserAttachChild(ps, node,
simpleExpr)) {
            parseTreeFree(node);
            return NULL;
        }
    }

    return node;
}

```

```
}
```

7. Pencocokan Token dan Konstruksi Parse Tree

```
static bool parserExpectToken(Parser *ps, TokenType expected,
ParseTreeNode *parent) {
    char found[128];
    Token *token;

    if (parserCurrentType(ps)==expected) {
        if (parent!=NULL) {
            ParseTreeNode *leaf;

            token=parserCurrentToken(ps);
            leaf=parserCreateTokenNode(ps, token);
            if (leaf==NULL) {
                return false;
            }

            if (!parserAttachChild(ps, parent, leaf)) {
                return false;
            }
        }

        parserAdvance(ps);
        return true;
    }

    token=parserCurrentToken(ps);
    if (token!=NULL) {
        describeToken(token, found, sizeof(found));
    }
    else {
        (void)snprintf(found, sizeof(found), "eof");
    }

    parserRecordSyntaxErrorToken(
        ps,
        token,
        "Expected %s, found %s.",
        tokenTypeToString(expected),
        found
    );

    if (parserShouldInsertMissingToken(ps, expected)) {
        (void)parent;
        return true;
    }

    if (parserCurrentType(ps)!=TOKEN_EOF) {
        parserAdvance(ps);
    }

    if (parserCurrentType(ps)==expected) {
```

```

        if (parent!=NULL) {
            ParseTreeNode *leaf=parserCreateTokenNode(ps,
parserCurrentToken(ps));
            if (leaf==NULL) {
                return false;
            }

            if (!parserAttachChild(ps, parent, leaf)) {
                return false;
            }
        }

        parserAdvance(ps);
    }

    return true;
}

```

8. Informasi Kesalahan

```

static void parserRecordSyntaxErrorToken(Parser *ps, Token *token, const
char *fmt, ...) {
    va_list args;

    va_start(args, fmt);
    parserRecordSyntaxErrorV(ps, token, fmt, args);
    va_end(args);
}

```

```

parserRecordSyntaxErrorToken(
    ps,
    token,
    "Expected %s, found %s.",
    tokenTypeToString(expected),
    found
);

```

9. Validasi terhadap Tata Bahasa Formal

```

static bool validateWithGrammar(Parser *ps, ParseTreeNode *root) {
    Node *grammarRoot;
    bool complete;

    grammarRoot=convertParseTreeToGrammarNode(ps, root);
    if (grammarRoot==NULL) {
        return false;
    }

    complete=is_program_complete(grammarRoot);
    grammarFreeNode(grammarRoot);
}

```

```

        if (!complete) {
            return parserSetError(ps, "Grammar validation failed: parse tree
does not match the grammar rules.");
        }

        return true;
    }
}

```

E. Main

Main mengalami penyesuaian untuk mengakomodasi penambahan tahap syntax analysis. Perubahan utama yang dilakukan adalah penambahan opsi untuk menjalankan syntax analysis serta penambahan fungsi runSyntaxAnalysis.

```

static void runSyntaxAnalysis(void) {
    char inputPath[IO_MAX_PATH];
    char outputPath[IO_MAX_PATH];
    FILE *out;
    SyntaxResult syntaxResult;

    if (!promptSyntaxPaths(inputPath, sizeof(inputPath), outputPath,
sizeof(outputPath))) {
        return;
    }

    if (analyzeSyntaxFile(inputPath, &syntaxResult)) {
        printf("\nSyntax analysis success.\n");
        printf("%s\n\n", syntaxResult.message);
        printf("Parse tree:\n");
        parseTreePrint(syntaxResult.tree, stdout);
        printf("\n");

        out=fopen(outputPath, "w");
        if (out==NULL) {
            fprintf(stderr, "Gagal membuat file output: %s\n\n",
outputPath);
            freeSyntaxResult(&syntaxResult);
            return;
        }

        parseTreePrint(syntaxResult.tree, out);
        fprintf(out, "\n");
        fclose(out);
        printf("Output parse tree tersimpan di: %s\n\n", outputPath);

        freeSyntaxResult(&syntaxResult);
        return;
    }

    fprintf(stderr, "\nSyntax analysis failed.\n");
    fprintf(stderr, "%s\n\n", syntaxResult.message);
    freeSyntaxResult(&syntaxResult);
}

```

```

}

static bool promptAnalysisMode(int *mode) {
    printf("Pilih mode analisis:\n");
    printf("1. Lexical Analysis\n");
    printf("2. Syntax Analysis\n");
    printf("0. Exit\n");
    printf("Masukkan pilihan: ");

    if (scanf("%d", mode)!=1) {
        fprintf(stderr, "Input pilihan harus berupa angka.\n\n");
        clearInputBuffer();
        return false;
    }

    return true;
}

int main(void) {
    int mode;

    while (true) {
        if (!promptAnalysisMode(&mode)) {
            continue;
        }

        if (mode==0) {
            printf("Program dihentikan.\n");
            return EXIT_SUCCESS;
        }

        if (mode==1) {
            runLexicalAnalysis();
        }
        else if (mode==2) {
            runSyntaxAnalysis();
        }
        else {
            fprintf(stderr, "Pilihan tidak valid. Gunakan 0, 1, atau
2.\n\n");
        }
    }
}

```


Pengujian

Testing 1

Keterangan Input : Input program arion normal

Input
<pre>program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>
Output
<pre><program> -- <program-header> -- programsy -- ident(Hello) `-- semicolon  -- <declaration-part>     `-- <var-declaration> -- varsy -- <identifier-list> -- ident(a) -- comma `-- ident(b)              -- colon              -- <type>                 `-- ident(integer)</pre>

```

|      `-- semicolon
|-- <compound-statement>
|  |-- beginsy
|  |-- <statement-list>
|  |  |-- <statement>
|  |  |  `-- <assignment-statement>
|  |  |      |-- <variable>
|  |  |      |  `-- ident(a)
|  |  |      |-- becomes
|  |  |      `-- <expression>
|  |  |          `-- <simple-expression>
|  |  |              `-- <term>
|  |  |                  `-- <factor>
|  |  |                      `-- intcon(5)
|  |  |-- semicolon
|  |  |-- <statement>
|  |  |  `-- <assignment-statement>
|  |  |      |-- <variable>
|  |  |      |  `-- ident(b)
|  |  |      |-- becomes
|  |  |      `-- <expression>
|  |  |          `-- <simple-expression>
|  |  |              |-- <term>
|  |  |                  |  `-- <factor>
|  |  |                      |  `-- <variable>
|  |  |                          `-- ident(a)
|  |  |                  |-- <additive-operator>
|  |  |                      |  `-- plus
|  |  |                      `-- <term>
|  |  |                          `-- <factor>
|  |  |                              `-- intcon(10)

```

<pre> -- semicolon -- <statement> `-- <procedure/function-call>                       -- ident(writeln)                       -- lparent                       -- <parameter-list>                              -- <expression>                                    `-- <simple-expression> -- <term> -- <factor> -- string(Result =) -- comma -- <expression> `-- <simple-expression> -- <term> -- <factor> -- <variable> -- ident(b) -- rparent -- semicolon -- endsy -- period </pre>	
Penjelasan	Program sederhana yang deklarasi dua variabel integer, melakukan assignment, ekspresi aritmatika penjumlahan, dan memanggil prosedur writeln. Testcase ini memverifikasi bahwa parser dapat memproses struktur program paling dasar: program-header, var-declaration, dan compound-statement berisi beberapa assignment-statement serta procedure/function-call dengan parameter-list berisi string literal dan variabel.

Testing 2

Keterangan Input : Input Array & Record Types

Input
<pre>program ArrayRecordTest; type IntArray == array[1..5] of integer; CharRange == array['a'..'z'] of integer; Point == record x: integer; y: integer end; var arr: IntArray; p: Point; i: integer; begin arr[1] := 42; arr[3] := arr[1] + 10; p.x := 100; p.y := 200; i := p.x + arr[1] end.</pre>
Output
<pre><program> -- <program-header> -- programsy -- ident(ArrayRecordTest)</pre>

```

|    `-- semicolon
|-- <declaration-part>
|    |-- <type-declaration>
|    |    |-- typesy
|    |    |-- ident(IntArray)
|    |    |-- eql
|    |    |-- <type>
|    |    |    `-- <array-type>
|    |    |        |-- arraysy
|    |    |        |-- lbrack
|    |    |        |-- <range>
|    |    |        |    |-- <constant>
|    |    |        |    |    `-- intcon(1)
|    |    |        |    |-- period
|    |    |        |    |-- period
|    |    |        |    `-- <constant>
|    |    |        |        `-- intcon(5)
|    |    |        |-- rbrack
|    |    |        |-- ofsy
|    |    |        `-- <type>
|    |    |            `-- ident(integer)
|    |    |-- semicolon
|    |    |-- ident(CharRange)
|    |    |-- eql
|    |    |-- <type>
|    |    |    `-- <array-type>
|    |    |        |-- arraysy
|    |    |        |-- lbrack
|    |    |        |-- <range>
|    |    |        |    |-- <constant>
|    |    |        |    |    `-- charcon(a)

```

```

|   |   |   |   |   |-- period
|   |   |   |   |   |-- period
|   |   |   |   |   `-- <constant>
|   |   |   |   |       `-- charcon(z)
|   |   |   |   |-- rbrack
|   |   |   |   |-- ofsy
|   |   |   |   `-- <type>
|   |   |   |       `-- ident(integer)
|   |   |-- semicolon
|   |   |-- ident(Point)
|   |   |-- eql
|   |   |-- <type>
|   |   |   `-- <record-type>
|   |   |       |-- recordsy
|   |   |       |-- <field-list>
|   |   |       |   |-- <field-part>
|   |   |       |   |   |-- <identifier-list>
|   |   |       |   |   |   `-- ident(x)
|   |   |       |   |   |-- colon
|   |   |       |   |   `-- <type>
|   |   |       |   |       `-- ident(integer)
|   |   |       |   |-- semicolon
|   |   |       |   `-- <field-part>
|   |   |       |       |-- <identifier-list>
|   |   |       |       |   `-- ident(y)
|   |   |       |       |-- colon
|   |   |       |       `-- <type>
|   |   |       |           `-- ident(integer)
|   |   |       `-- endsy
|   |   `-- semicolon
|   `-- <var-declaration>

```

```

|      |-- varsy
|      |-- <identifier-list>
|      |   |-- ident(arr)
|      |-- colon
|      |-- <type>
|      |   |-- ident(IntArray)
|      |-- semicolon
|      |-- <identifier-list>
|      |   |-- ident(p)
|      |-- colon
|      |-- <type>
|      |   |-- ident(Point)
|      |-- semicolon
|      |-- <identifier-list>
|      |   |-- ident(i)
|      |-- colon
|      |-- <type>
|      |   |-- ident(integer)
|      |-- semicolon
|-- <compound-statement>
|   |-- beginsy
|   |-- <statement-list>
|   |   |-- <statement>
|   |   |   |-- <assignment-statement>
|   |   |   |   |-- <variable>
|   |   |   |   |   |-- ident(arr)
|   |   |   |   |   |-- <component-variable>
|   |   |   |   |   |-- lbrack
|   |   |   |   |   |-- <index-list>
|   |   |   |   |   |   |-- intcon(1)
|   |   |   |   |   |-- rbrack

```



```

|   |   |           |-- <term>
|   |   |           |-- <factor>
|   |   |           |-- intcon(10)
|   |   |-- semicolon
|   |   |-- <statement>
|   |   |-- <assignment-statement>
|   |   |   |-- <variable>
|   |   |   |   |-- ident(p)
|   |   |   |   |-- <component-variable>
|   |   |   |   |-- period
|   |   |   |   |-- ident(x)
|   |   |   |-- becomes
|   |   |   |-- <expression>
|   |   |       |-- <simple-expression>
|   |   |       |-- <term>
|   |   |       |-- <factor>
|   |   |       |-- intcon(100)
|   |   |-- semicolon
|   |   |-- <statement>
|   |   |-- <assignment-statement>
|   |   |   |-- <variable>
|   |   |   |   |-- ident(p)
|   |   |   |   |-- <component-variable>
|   |   |   |   |-- period
|   |   |   |   |-- ident(y)
|   |   |   |-- becomes
|   |   |   |-- <expression>
|   |   |       |-- <simple-expression>
|   |   |       |-- <term>
|   |   |       |-- <factor>
|   |   |       |-- intcon(200)

```

```

|   |   |-- semicolon
|   |   `-- <statement>
|   |       |-- <assignment-statement>
|   |           |-- <variable>
|   |           |   |-- ident(i)
|   |           |-- becomes
|   |           |-- <expression>
|   |               |-- <simple-expression>
|   |                   |-- <term>
|   |                       |   |-- <factor>
|   |                           |   |-- <variable>
|   |                               |   |-- ident(p)
|   |                                   |--
|   |                                   <component-variable>
|   |                                       |   |-- period
|   |                                       |   |-- ident(x)
|   |                                       |-- <additive-operator>
|   |                                           |-- plus
|   |                                           |-- <term>
|   |                                               |-- <factor>
|   |                                                   |-- <variable>
|   |                                                       |-- ident(arr)
|   |                                                           |--
|   |                                                           <component-variable>
|   |                                                               |-- lbrack
|   |                                                               |-- <index-list>
|   |                                                                   |-- intcon(1)
|   |                                                                       |-- rbrack
|   | `-- endsy
|-- period

```

Penjelasan	Program yang mendefinisikan tipe array dengan indeks integer (array[1..5]), array dengan indeks karakter (array['a'..'z']), dan tipe record dua field. Testcase ini memverifikasi node <array-type> dengan <range> berbasis intcon maupun charcon, node <record-type> dengan <field-list> dan <field-part>, serta <component-variable> berupa akses elemen array (arr[1]) dan akses field record (p.x, p.y) di sisi kiri maupun kanan assignment.
-------------------	--

Testing 3

Keterangan Input : Input program procedure and function

Input
<pre> program SubprogramTest; var result: integer; procedure printValue(x: integer); begin writeln(x) end; function add(a, b: integer): integer; begin add := a + b end; function multiply(a, b: integer): integer; var temp: integer; begin </pre>

```

    temp := a * b;
    multiply := temp
end;

begin
    result := add(3, 4);
    result := multiply(result, 2);
    printValue(result)
end.

```

Output

```

<program>
|-- <program-header>
|   |-- programsy
|   |-- ident(SubprogramTest)
|   `-- semicolon
|-- <declaration-part>
|   |-- <var-declaration>
|   |   |-- varsy
|   |   |-- <identifier-list>
|   |   |   |-- ident(result)
|   |   |-- colon
|   |   |-- <type>
|   |   |   |-- ident(integer)
|   |   `-- semicolon
|   |-- <subprogram-declaration>
|   |   |-- <procedure-declaration>
|   |   |   |-- proceduresy
|   |   |   |-- ident(printValue)
|   |   |   |-- <formal-parameter-list>
|   |   |   |   |-- lparent

```

```

|      |      |      |-- <parameter-group>
|      |      |      |-- <identifier-list>
|      |      |      |      `-- ident(x)
|      |      |      |-- colon
|      |      |      |      `-- ident(integer)
|      |      |      `-- rparent
|      |      |-- semicolon
|      |      |-- block
|      |      |-- <declaration-part>
|      |      |      `-- <compound-statement>
|      |      |      |-- beginsy
|      |      |      |-- <statement-list>
|      |      |      |      `-- <statement>
|      |      |      |      |      `-- <procedure/function-call>
|      |      |      |      |      |-- ident(writeln)
|      |      |      |      |      |-- lparent
|      |      |      |      |      |-- <parameter-list>
|      |      |      |      |      |      `-- <expression>
|      |      |      |      |      |      `--
<simple-expression>
|      |      |      |      |      |      `-- <term>
|      |      |      |      |      |      `--
<factor>
|      |      |      |      |      |      `--
<variable>
|      |      |      |      |      |
`-- ident(x)
|      |      |      |      |      |      `-- rparent
|      |      |      |      |      |      `-- endsy
|      |      |      |      |      |      `-- semicolon
|      |-- <subprogram-declaration>

```

		`-- <function-declaration>			
		-- functionsy			
		-- ident(add)			
		-- <formal-parameter-list>			
			-- lparent		
			-- <parameter-group>		
				-- <identifier-list>	
					-- ident(a)
					-- comma
					`-- ident(b)
					-- colon
					`-- ident(integer)
					`-- rparent
		-- colon			
		-- ident(integer)			
		-- semicolon			
		-- block			
			-- <declaration-part>		
			`-- <compound-statement>		
			-- beginsy		
			-- <statement-list>		
				`-- <statement>	
				`-- <assignment-statement>	
				-- <variable>	
					`-- ident(add)
					-- becomes
					`-- <expression>
					`--
<simple-expression>					
					-- <term>
					`-- <factor>

[illegible]

				-- <factor>
				--
<variable>				
				--
ident(b)				
				-- semicolon
				-- <statement>
				-- <assignment-statement>
				-- <variable>
				-- ident(multiply)
				-- becomes
				-- <expression>
				--
<simple-expression>				
				-- <term>
				-- <factor>
				--
<variable>				
				--
ident(temp)				
				-- endsy
				-- semicolon
-- <compound-statement>				
				-- beginsy
				-- <statement-list>
				-- <statement>
				-- <assignment-statement>
				-- <variable>
				-- ident(result)
				-- becomes
				-- <expression>

```

|   |   |           `-- <simple-expression>
|   |   |           `-- <term>
|   |   |           `-- <factor>
|   |   |           `--
<procedure/function-call>
|   |   |           |-- ident(add)
|   |   |           |-- lparent
|   |   |           |-- <parameter-list>
|   |   |           |   |-- <expression>
|   |   |           |   |   `--
<simple-expression>
|   |   |           |   |   `-- <term>
|   |   |           |   |   `--
<factor>
|   |   |           |   |
|-- intcon(3)
|   |   |           |   |-- comma
|   |   |           |   |-- <expression>
|   |   |           |   |   `--
<simple-expression>
|   |   |           |   |   `-- <term>
|   |   |           |   |   `--
<factor>
|   |   |           |
|-- intcon(4)
|   |   |           |-- rparent
|   |   |-- semicolon
|   |   |-- <statement>
|   |   |-- <assignment-statement>
|   |   |   |-- <variable>
|   |   |   |   |-- ident(result)

```

```

|   |   |   |-- becomes
|   |   |   `-- <expression>
|   |   |       `-- <simple-expression>
|   |   |           `-- <term>
|   |   |               `-- <factor>
|   |   |                   `--
<procedure/function-call>
|   |   |   |-- ident(multiply)
|   |   |   |-- lparent
|   |   |   |-- <parameter-list>
|   |   |   |   |-- <expression>
|   |   |   |   |   `--
<simple-expression>
|   |   |   |   |   `-- <term>
|   |   |   |   |       `--
<factor>
|   |   |   |   |
|-- <variable>
|   |   |   |   |
|-- ident(result)
|   |   |   |   |   |-- comma
|   |   |   |   |   |-- <expression>
|   |   |   |   |       `--
<simple-expression>
|   |   |   |   |   `-- <term>
|   |   |   |   |       `--
<factor>
|   |   |   |   |
|-- intcon(2)
|   |   |   |   |-- rparent
|   |   |   |-- semicolon

```

<pre> `-- <statement>                   `-- <procedure/function-call> -- ident(printValue) -- lparent -- <parameter-list> `-- <expression>                                  `-- <simple-expression> `-- <term>                                          `-- <factor> `-- <variable>                                                  `-- ident(result) `-- rparent        `-- endsy -- period </pre>	
Penjelasan	Program yang mendefinisikan satu prosedur dan dua fungsi. Testcase ini memverifikasi <procedure-declaration> dengan <formal-parameter-list>, <function-declaration> dengan tipe kembalian, serta <block> yang mengandung <var-declaration> lokal (pada fungsi multiply). Di bagian utama, pemanggilan fungsi muncul sebagai <factor> dalam ekspresi (sisi kanan assignment), yang menunjukkan bahwa <procedure/function-call> dapat bersarang di dalam <expression>.

Testing 4

Keterangan Input : Input program Control Flow

Input
<pre> program ControlFlowTest; var </pre>

```

    i, x, sum: integer;
begin
    x := 0;
    if x == 0 then
        x := 1
    else
        x := 2;
    sum := 0;
    while x < 10 do
        x := x + 1;
    for i := 1 to 5 do
        sum := sum + i;
    for i := 10 downto 1 do
        writeln(i)
    end.

```

Output

```

<program>
|-- <program-header>
|   |-- programsy
|   |-- ident(ControlFlowTest)
|   `-- semicolon
|-- <declaration-part>
|   `-- <var-declaration>
|       |-- varsy
|       |-- <identifier-list>
|           |   |-- ident(i)
|           |   |-- comma
|           |   |-- ident(x)
|           |   |-- comma
|           `-- ident(sum)

```

```

|      |-- colon
|      |-- <type>
|      |    `-- ident(integer)
|      `-- semicolon
|-- <compound-statement>
|  |-- beginsy
|  |-- <statement-list>
|  |  |-- <statement>
|  |  |    `-- <assignment-statement>
|  |  |      |-- <variable>
|  |  |      |    `-- ident(x)
|  |  |      |-- becomes
|  |  |      `-- <expression>
|  |  |          `-- <simple-expression>
|  |  |              `-- <term>
|  |  |                  `-- <factor>
|  |  |                      `-- intcon(0)
|  |  |-- semicolon
|  |  |-- <statement>
|  |  |    `-- <if-statement>
|  |  |      |-- ifsy
|  |  |      |-- <expression>
|  |  |      |    |-- <simple-expression>
|  |  |      |    |    `-- <term>
|  |  |      |    |    `-- <factor>
|  |  |      |    |    `-- <variable>
|  |  |      |    |    `-- ident(x)
|  |  |      |    |-- <relational-operator>
|  |  |      |    |    `-- eql
|  |  |      |    `-- <simple-expression>
|  |  |      |        `-- <term>

```

```

|-- <factor>
|-- intcon(0)
|-- thensy
|-- <statement>
|-- <assignment-statement>
|-- <variable>
|-- ident(x)
|-- becomes
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- intcon(1)
|-- elsesy
|-- <statement>
|-- <assignment-statement>
|-- <variable>
|-- ident(x)
|-- becomes
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- intcon(2)
|-- semicolon
|-- <statement>
|-- <assignment-statement>
|-- <variable>
|-- ident(sum)
|-- becomes
|-- <expression>

```

```

|   |   |           `-- <simple-expression>
|   |   |           `-- <term>
|   |   |           `-- <factor>
|   |   |           `-- intcon(0)
|   |   |-- semicolon
|   |   |-- <statement>
|   |   `-- <while-statement>
|   |       |-- whilesy
|   |       |-- <expression>
|   |           |-- <simple-expression>
|   |               |-- <term>
|   |                   |-- <factor>
|   |                       |-- <variable>
|   |                           |-- ident(x)
|   |           |-- <relational-operator>
|   |               |-- lss
|   |           `-- <simple-expression>
|   |               `-- <term>
|   |                   `-- <factor>
|   |                       `-- intcon(10)
|   |       |-- dosy
|   |       `-- <statement>
|   |           `-- <assignment-statement>
|   |               |-- <variable>
|   |                   |-- ident(x)
|   |               |-- becomes
|   |               `-- <expression>
|   |                   `-- <simple-expression>
|   |                       |-- <term>
|   |                           |-- <factor>
|   |                               `-- <variable>

```



```
|      |      |      |      |-- ident(x)
|      |      |      |      |-- <additive-operator>
|      |      |      |      |-- plus
|      |      |      |      |-- <term>
|      |      |      |      |-- <factor>
|      |      |      |      |-- intcon(1)
|      |      |-- semicolon
|      |      |-- <statement>
|      |      |-- <for-statement>
|      |      |-- forsy
|      |      |-- ident(i)
|      |      |-- becomes
|      |      |-- <expression>
|      |      |-- <simple-expression>
|      |      |-- <term>
|      |      |-- <factor>
|      |      |-- intcon(1)
|      |      |-- tosy
|      |      |-- <expression>
|      |      |-- <simple-expression>
|      |      |-- <term>
|      |      |-- <factor>
|      |      |-- intcon(5)
|      |      |-- dosy
|      |      |-- <statement>
|      |      |-- <assignment-statement>
|      |      |-- <variable>
|      |      |-- ident(sum)
|      |      |-- becomes
|      |      |-- <expression>
|      |      |-- <simple-expression>
```

```

|-- <term>
|   |-- <factor>
|       |-- <variable>
|           |-- ident(sum)
|-- <additive-operator>
|   |-- plus
|-- <term>
|       |-- <factor>
|           |-- <variable>
|               |-- ident(i)
|-- semicolon
|-- <statement>
|   |-- <for-statement>
|       |-- forsy
|       |-- ident(i)
|       |-- becomes
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|                   |-- <factor>
|                       |-- intcon(10)
|       |-- downtosy
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|                   |-- <factor>
|                       |-- intcon(1)
|       |-- dosy
|-- <statement>
|   |-- <procedure/function-call>
|       |-- ident(writeln)

```

		-- lparent
		-- <parameter-list>
		`-- <expression>
		`-- <simple-expression>
		`-- <term>
		`-- <factor>
		`-- <variable>
		`--
ident(i)		
		-- rparent
		`-- endsy
-- period		
Penjelasan	Program yang menguji empat bentuk statement kontrol sekaligus. Testcase ini memverifikasi <if-statement> dengan cabang else, <while-statement> dengan kondisi relasional, <for-statement> dengan arah to (naik), dan <for-statement> dengan arah downto (turun). Setiap statement menggunakan ekspresi yang berbeda sehingga seluruh operator relasional dasar (==, <) turut terlatih.	

Testing 5

Keterangan Input : Input missing semicolon

Input
<pre> program MissingSemicolon; var a: integer b: integer; begin a := 1; </pre>

<pre> b := 2 end. </pre>	
Output	
<pre> Syntax analysis failed. Ditemukan error syntax: Line 4 near ident (b): Expected semicolon, found ident (b) . </pre>	
Penjelasan	Program dengan deklarasi variabel a: integer yang tidak diakhiri ; sebelum deklarasi b: integer berikutnya. Error yang dihasilkan: Line 4 near ident (b): Expected semicolon, found ident (b). Parser mendeteksi bahwa setelah membaca <identifier-list> colon <type> untuk variabel a, token berikutnya yang ditemukan adalah ident(b) bukan semicolon, sehingga produksi var-declaration gagal.

Testing 6

Keterangan Input : Input program missing period

Input
<pre> program MissingPeriod; var x: integer; begin x := 42 end </pre>
Output
<pre> Syntax analysis failed. </pre>

Ditemukan error syntax: Line 7 near eof: Expected period, found eof.	
Penjelasan	Program yang diakhiri end tanpa tanda titik . sebagai penutup. Error yang dihasilkan: Line 7 near eof: Expected period, found eof. Parser berhasil memproses seluruh compound-statement, namun token terakhir yang diharapkan adalah period sesuai produksi <program>, sedangkan yang ditemukan adalah eof.

Testing 7

Keterangan Input : Input program missing then

Input	
<pre> program MissingThen; var x: integer; begin x := 5; if x > 0 x := 1 end. </pre>	
Output	
<pre> Syntax analysis failed. Ditemukan error syntax: Line 7 near ident (x): Expected then, found ident (x). </pre>	
Penjelasan	Program dengan if-statement yang melewati kata kunci then antara kondisi dan body. Error yang dihasilkan:

	Line 7 near ident (x): Expected thensy, found ident (x). Parser berhasil memproses ifsy dan <expression> (x > 0), namun token berikutnya yang wajib ada adalah thensy sesuai produksi <if-statement>. Yang ditemukan justru ident(x) dari baris body statement.
--	--

Testing 8

Keterangan Input : Input program empty statement

Input	
<pre> program DoublesSemicolon; var x: integer; begin x := 1; ; x := x + 1 end. </pre>	
Output	
<pre> Syntax analysis failed. Ditemukan error syntax: Line 6 near semicolon: Expected statement after semicolon. </pre>	
Penjelasan	Program dengan dua tanda semicolon berturutan (;) di dalam compound-statement, di mana semicolon kedua tidak didahului statement apapun. Error yang dihasilkan: Line 6 near semicolon: Expected statement after semicolon. Parser memproses statement pertama (x := 1), lalu menemukan ;,

	kemudian mengharapkan statement berikutnya. Yang ditemukan adalah ; lagi (bukan awal statement yang valid), sehingga parser melaporkan error.
--	---

Kesimpulan dan Saran

Kesimpulan

Program berhasil dirancang dan diimplementasikan dengan baik, dan source code berhasil untuk membangun parse tree dengan benar berdasarkan input token dari lexical analysis terhadap suatu test case. Syntax analysis berhasil diimplementasikan, dan output parse tree yang dibuat sudah mengikuti rancangan grammar yang sudah diimplementasikan ke dalam source code program. Program juga berhasil untuk menunjukkan error ketika bertemu dengan test case token yang tidak mengikuti rancangan aturan grammar language dengan benar. Secara keseluruhan, syntax analysis berhasil untuk diimplementasikan ke dalam source code program.

Saran

Spesifikasi tugas besar tidak terlalu berat dan tergolong bisa dinikmati pengerjaannya, serta spesifikasi masih sesuai dengan cakupan materi yang diajarkan di kelas.

Semoga pada milestone 2 spesifikasi dapat diperiksa lebih dalam kembali dan diperjelas, agar tidak terdapat terlalu banyak revisi terhadap spesifikasi tugas baik dalam dokumen spesifikasi ataupun lewat QnA Milestone 2. Selain itu, spesifikasi milestone 2 dianggap sudah sesuai dengan kemampuan dan pengetahuan mahasiswa yang didapatkan dari kelas.

Lampiran

Link Repositori

<https://github.com/RayapSunggal/SHT-Tubes-IF2224-2026>

Pembagian Tugas

Nama	Tugas	Persentase
Audric Y. M.Simatupang - 13524010	Testing source code, Pembuatan Laporan	25%
Raynard Fausta - 13524052	Pembuatan source code	25%
Nathan E.C Marpaung - 13524062	Testing source code, Pembuatan Laporan	25%
Richard S Simanullang - 13524112	Bugfixing dan revisi source code, testing source code, pembuatan hasil test case pada laporan	25%

Referensi

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. 2007. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Boston: Pearson Addison-Wesley.

Hopcroft, J. E., Motwani, R., & Ullman, J. D. 2006. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Upper Saddle River, NJ: Pearson Prentice Hall.

Stanford University. n.d. *CS143 Handout: Lexical Analysis*. Stanford CS143

TutorialsPoint. n.d. *Compiler Design - Lexical Analysis*. Tersedia pada:

https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm

Edunex n.d. *Compiler and Recursive Descent Parser*. Tersedia pada:

https://edunexcontentprodhot.blob.core.windows.net/edunex/2026/86183-Formal-Language-Theory-and-Automata/434820-Compiler-dan-Recursive-Descent-Parser/file/1776038658130_Compiler-and-Recursive-Descent-Parser?sv=2024-11-04&spr=https&st=2026-04-13T00%3A04%3A18Z&se=2028-04-12T00%3A04%3A18Z&sr=b&sp=r&sig=w8T8grhs%2Bk%2Bjw0DTW3wy53c%2B0oXT%2Fm88IHauSRTIPPs%3D&rsct=application%2Fpdf